Reviewer Pack — VC dimension of gadget row family tightly bounds lifted D^cc

Entry 02a2f4e161dc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 10:22:54 UTC

VC dimension of gadget row family tightly bounds lifted D^cc

Entry ID: `02a2f4e161dc` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 10:22:54 UTC

1. Conjecture

Field A (mathematical branch): Vapnik-Chervonenkis combinatorial dimension theory **Field B** (complexity object): Query-to-communication lifting for deterministic two-party protocols (Raz-McKenzie gadget composition)

Statement:

> Let g: A × B → {0,1} be a gadget and let R_g = {g(a,·) : a ∈ A} ⊆ 2^B be its row family, with VC dimension d = VC(R_g). For every Boolean function f : {0,1}^n → {0,1} with deterministic decision-tree depth D(f) ≥ 2, the deterministic communication complexity of the Raz-McKenzie lift satisfies D^cc(f ∘ g^n) ≥ (1/2)·D(f)·d − n. Equivalently, log_2 rank_{GF(2)}(M_{f∘g^n}) ≥ (1/2)·D(f)·VC(R_g) − n for every (f,g).

Rationale (proposer's reasoning):

> Standard lifting theorems give the multiplier log b for index_b, where b is also exactly VC(R_{index_b}); for inner-product gadgets the same equality holds. We conjecture that across ALL gadgets the per-coordinate information cost of distinguishing rows in any protocol is governed not by |A| or by rank but by the shatter dimension of R_g, because every protocol leaf must combinatorially separate a shattered family. This bridges PAC-style combinatorial dimension with the structural side of lifting and yields a falsifier if any low-VC gadget achieves anomalously high D^cc.

Taxonomy category: `LIFTING` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `d967702cbf848c57`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across 30 seeds sampling (f,g) per the strategy, the conjecture is SUPPORTED iff at least 90% of (f,g) pairs satisfy log2(rank_GF2(M_{f∘g^n})) >= 0.5*D(f)*VC(R_g) - n, and the mean slack (log2 rank - (0.5*D(f)*VC(R_g) - n)) across all pairs is >= 0. Otherwise FALSIFIED.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Raz-McKenzie lifting gadget VC dimension communication complexity` - `query-to-communication lifting deterministic decision tree gadget combinatorial parameter` - `deterministic communication complexity lower bound lifted function VC dimension rank`

Top relevant hits considered: - [http://arxiv.org/abs/1503.07648v2] Sign rank versus VC dimension - [http://arxiv.org/abs/1904.13056v4] Query-to-Communication Lifting Using Low-Discrepancy Gadgets - [http://arxiv.org/abs/2001.02144v1] Lifting with Simple Gadgets and Applications to Circuit and Proof Complexity - [http://arxiv.org/abs/2512.11833v1] Soft Decision Tree classifier: explainable and extendable PyTorch implementation - [http://arxiv.org/abs/2211.17214v1] Lifting to Parity Decision Trees Via Stifling - [http://arxiv.org/abs/1806.02734v3] Spectral lower bounds for the orthogonal and projective ranks of a graph - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def VC(R):
        B_size = len(R[0])
        for d in range(1, 2**B_size + 1):
            if all(any(all(R[j][i] == R[k][i] for i in range(B_size)) for j in S) for k in range(l
                return d - 1
        return len(R)

    def log2_rank_GF2(M):
        n, m = len(M), len(M[0])
        rank = 0
        for i in range(n):
            if any(M[i][j] != 0 for j in range(m)):
                rank += 1
                for j in range(m):
```

```
                    M[i][j] /= M[i][j]
                for k in range(n):
                    if k != i and any(M[k][j] != 0 for j in range(m)):
                        for j in range(m):
                            M[k][j] -= M[i][j] * M[k][i]
        return rank

    def index_b(a, b):
        return a == b

    def inner_product_b(a, b):
        return sum(x * y for x, y in zip(a, b))

    def equality_gadget(a, b):
        return [int(i == j) for i, j in zip(a, b)]

    gadgets = {
        "index": index_b,
        "inner_product": inner_product_b,
        "equality": equality_gadget
    }

    results = []
    for n in {2, 3, 4}:
        for A_size in {4, 8}:
            for B_size in {2, 3, 4}:
                f = [random.choice([0, 1]) for _ in range(2**n)]
                g = random.choice(list(gadgets.values()))
                R_g = [[g(a, b) for b in range(B_size)] for a in range(A_size)]
                d = VC(R_g)

                M = [[f[int(''.join(map(str, g(a, b))), 2)] for b in range(B_size)] for _ in range
                rank = log2_rank_GF2(M)

                slack = rank - (0.5 * n * d - n)
                results.append({
                    "metric_name": "slack",
                    "metric_value": slack,
                    "instances_tested": 1,
                    "conjecture_holds": slack >= 0,
                    "counterexample": ""
                })

    mean_slack = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    return {
        "seed": seed,
        "mean_slack": mean_slack,
        "support_fraction": support_fraction
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
```

```python
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_slack = sum(result["mean_slack"] for result in results) / len(results)
    support_fraction = sum(result["support_fraction"] for result in results) / len(results)

    if all(result["support_fraction"] >= 0.9 for result in results):
        print(f"RESULT: SUPPORTED mean={mean_slack} std=NA support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"slack < 0\" first_failing_seed={first_failing_s
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ad3e622c.py", line 93, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ad3e622c.py", line 66, in run_trial
    M = [[f[int(''.join(map(str, g(a, b))), 2)] for b in range(B_size)] for _ in range(A_size**n)]
                                        ^
UnboundLocalError: cannot access local variable 'a' where it is not associated with a value
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed with an UnboundLocalError before producing any data, so no support fraction or slack statistics were computed. Per rule 2, a crashed test yields INCONCLUSIVE. | next: Fix the bug in the matrix construction (the inner comprehension iterates over '_' instead of binding 'a', so 'a' is unbound); enumerate a ∈ A^n properly via itertools.product, then rerun the 30-seed protocol.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	121076
2	preregistration	claude_max	opus	0	0	6365
3	novelty	claude_max	opus	0	0	3667
4	novelty	claude_max	opus	0	0	10193
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	41092
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15080
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14231
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10925
9	judge	claude_max	opus	0	0	5292

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 227921 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/02a2f4e161dc.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/02a2f4e161dc.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/02a2f4e161dc.tar.gz` (if generated)